SliceMatic

Ordering System — Stage 1 Submission

Product Requirements Document + Business Unit Economics

FDE Programme · Batch 2487 · PizzaFlow Applied Project
Stage 1 · 20 Points · Due: June 25, 2026

AI Feature: Option A — Personalised Recommendation Engine

TL;DR

> *SliceMatic needs a digital ordering system to replace phone intake at a single Delhi outlet. This PRD covers the full product: a Gradio MVP (Stage 2) that validates the ordering core, and a production Vercel + Supabase web app with an AI recommendation engine (Stage 3). Stage 2 is the RAT — it tests whether ordering logic, validation, pricing, and persistence work correctly. Stage 3 is the product. The AI feature is locked as* ***Option A: a personalised recommendation shown to returning customers before menu selection****, powered by Supabase order history via OpenRouter. This document is written precisely enough that a developer who has never seen the spec can build it correctly.*

What we are NOT building (any stage): real payment gateway, delivery address capture, inventory management, or mixed-pizza orders.

PART A — PRODUCT REQUIREMENTS DOCUMENT

A.1 Product Vision

SliceMatic's ordering system is a digital intake-to-bill platform that replaces phone ordering for a single-outlet pizza delivery brand in New Ashok Nagar, Delhi. It lets a customer self-serve the entire ordering journey — identity, quantity, menu selection, discount, GST, and payment — in a guided flow that computes every rupee deterministically and **persists each completed order as structured, analysable data**. It serves two users: the **customer**, who gets a fast, error-free order with a transparent bill, and the **owner**, who gets a queryable order database that surfaces best-sellers, peak hours, and repeat-customer behaviour. The system removes three failures of phone ordering: the single-line throughput ceiling at peak, manual pricing/GST errors, and the total absence of order data. By Stage 3 the system adds a personalised recommendation engine and an admin analytics dashboard.

A.2 Target Customer Personas

Persona 1 — The Customer. Urban millennial or Gen Z, 18–35, within 4 km of New Ashok Nagar. Orders pizza by phone today. Core pain: busy signal at peak, no digital receipt, no order history, no personalised experience. The recommendation engine turns order history into personalisation that aggregators cannot replicate.

Persona 2 — The New Customer (first order). Same demographic, no order history in the system. Needs a graceful path through the recommendation step without seeing an error or blank card. Treated as a distinct case because FR-18 explicitly handles it.

Persona 3 — The Admin (Rajan Sharma, the owner). Sole proprietor, non-technical. Currently has zero data on his business — no peak-hour visibility, no best-seller tracking, no repeat-customer identification. Needs a secure dashboard to view orders, filter by date and payment mode, understand what sells and when, and export data for his accountant.

A.3 User Stories

Customer-facing:

- As a customer, I want to place a pizza order digitally so I don't have to call and get a busy signal at peak.
- As a returning customer, I want a personalised recommendation based on my past orders so I can decide faster.
- As a new customer with no history, I want to go straight to the menu so the system doesn't block me.
- As a customer, I want an itemised bill with discount and GST so I know exactly what I'm paying.
- As a customer, I want specific error messages on invalid input so I know how to correct it without starting over.

Admin-facing:

• As the owner, I want a secure dashboard login so only I can see order data.

• As the owner, I want to filter orders by date and payment mode so I can reconcile cash, card, and UPI separately.

• As the owner, I want revenue, top pizza, and busiest hour so I can make staffing and menu decisions from data.

• As the owner, I want to export orders as CSV so I can share them with my accountant.

System/operator:

• As the operator, I want menu items stored in Supabase (Stage 3) so prices can be updated without touching code.

• As the operator, I want every order saved with the customer's phone number so the recommendation engine has data to query.

A.4 Functional Requirements

Stage 2 — Gradio MVP

#	Requirement	Detail
FR-1	Customer name	Alphabets and spaces only; min 2, max 40 chars. Reject empty, spaces-only, digits. Validate after strip().
FR-2	Customer phone	Exactly 10 digits; must start with 6, 7, 8, or 9 (Indian mobile format). Reject anything else.
FR-3	Session timestamp	Record an ISO-8601 timestamp at the start of every new order session.
FR-4	Quantity selection	Integer 1–10 inclusive. Reject 0, negatives, floats, non-numerics, and >10 with a specific message.
FR-5	Menu load at startup	Load all 3 menu files at app startup, before any user interaction. Format per line: ID;Name;Price. No hardcoded items.
FR-6	Menu display	Show each category as a numbered list with name + price in INR.
FR-7	Menu selection	Accept item number only. One base + one pizza + one topping. Qty N = N units of same combo. Reject out-of-range, letters, empty.
FR-8	Pricing engine	Line total = (base price + pizza price + topping price) x quantity.
FR-9	Discount logic	Auto-apply 10% when quantity >= 5. Show as explicit bill line. No discount for qty <= 4.
FR-10	GST calculation	18% GST on the post-discount subtotal. Menu prices are exclusive of GST.
FR-11	Itemised bill	Per-unit base/pizza/topping, qty, unit price, subtotal, discount (if any), GST, final payable — aligned, with currency symbol. Rendered in gr.Dataframe or gr.HTML.
FR-12	Payment flow	Exactly 3 modes: Cash, Card, UPI. Mode-specific confirmation message. Reject invalid selection and re-prompt.
FR-13	Order persistence	Append to orders_log.txt. Create if absent; append if exists. One order per block, pipe-separated, blank line between orders.

Stage 3 — Production Web App

#	Requirement	Detail
FR-14	Supabase schema	3 tables: menu_items (id, category, name, price, is_active), orders (id, customer_name, phone, quantity, subtotal, discount_amount, gst_amount, total, payment_mode, created_at), order_line_items (id, order_id FK, item_category, item_name, unit_price).
FR-15	Menu from DB	Stage 3 loads menu from menu_items table. Category field (base/pizza/topping) drives display. Not text files.
FR-16	Persist to Supabase	Every completed order writes to orders and order_line_items. Phone number is the customer identifier for history lookup. All Stage 2 logic preserved.
FR-17	AI recommendation — returning	After phone validated, query Supabase for last 10 orders by phone. If results exist, send to OpenRouter with system prompt. Show recommended pizza + topping + 1-2 sentence reason before menu selection. Customer can accept or skip.
FR-18	AI recommendation — new	If no order history found, skip recommendation step silently and proceed to menu selection. No error shown.
FR-19	AI recommendation — fallback	If OpenRouter returns null, errors, or times out (>5s): display fallback message and proceed. Do not block the flow. Log the failure.
FR-20	Admin auth	Admin login via Supabase Auth. Only authenticated admin can access dashboard. Non-admin redirected to login.
FR-21	Admin order history	All orders in a table. Filterable by date range and payment mode.
FR-22	Admin metrics	Total revenue, top-selling pizza by volume, busiest hour of day (from created_at).
FR-23	Admin CSV export	Export button downloads all currently filtered orders as CSV with all orders table fields and header row.
FR-24	Vercel deployment	Frontend on Vercel with public URL. Live and functional on July 2. Full ordering flow end-to-end.

A.5 AI Feature Specification — Recommendation Engine (Option A)

Trigger: after phone number is validated, before menu selection is shown.

Flow:

1. Query Supabase for the customer's last 10 orders by phone number.
2. No results found → skip to menu (FR-18).
3. Results found → format as pizza + topping pairs → send to OpenRouter with system prompt below → parse JSON response → display recommendation card.
4. Customer can accept (pre-fills menu selection) or skip (proceeds to full menu unaffected).

System prompt (documented verbatim in README):

```
"You are a recommendation engine for SliceMatic, a pizza delivery outlet in Delhi.
Based on the customer's past orders listed below, recommend ONE pizza and ONE topping
they haven't tried recently — or their clear favourite if they consistently reorder it.
Be brief and friendly, one or two sentences max.
Respond only as JSON: {pizza: string, topping: string, reason: string}.
Past orders: [order_history]"
```

Model selection: to be finalised before Stage 3 build. Recommend starting with a lightweight model on OpenRouter (e.g. Mistral 7B Instruct) for cost and latency. Choice must be documented with rationale in the

README as required by the rubric.

A.6 Non-Functional Requirements

Input validation rules:

- Name: regex ^[A-Za-z]{2,40}$ after strip(); reject if blank after strip.
- Phone: regex ^[6-9][0-9]{9}$.
- Quantity: must parse as int; reject '2.5', 'three', '', 0, 11+.
- Menu selection: must be an integer within 1..len(menu).

Error messages — specific and actionable, never a stack trace:

Field	Error Message
Name	Name must be 2–40 letters (spaces allowed), no numbers.
Phone	Enter a 10-digit Indian mobile starting with 6, 7, 8, or 9.
Quantity (range)	Quantity must be a whole number from 1 to 10.
Quantity (>10)	Maximum order is 10 pizzas per order.
Menu selection	Pick a number from the list (1–N).

Orders log format (Stage 2 — pipe-separated, one block per order, blank line between orders):

```
2025-06-23T19:42:11 | Rajan Sharma | 9876543210 | Base:Cheese Burst@229 |
Pizza:BBQ Chicken@379 | Topping:Extra Cheese@69 | Qty:5 | Subtotal:3385.00 |
Discount:338.50 | GST:548.37 | Total:3594.87 | Pay:UPI
```

Graceful failure modes:

- Missing menu file at startup → clear error naming the file, exit before any user prompt.
- Malformed line (missing field / non-numeric price) → skip that line, keep valid lines, log warning.
- Empty menu after parsing → refuse to start; tell the operator which file is empty.
- orders_log.txt does not exist on first run → create file; proceed normally.
- orders_log.txt write fails → confirm order to customer; warn operator; do not crash.
- OpenRouter API failure (Stage 3) → display fallback message per FR-19; proceed to menu.

A.7 Acceptance Criteria (Given / When / Then)

FR	Scenario	Given / When / Then
FR-1	Name — spaces only	Given user enters ' ', when they submit, then system shows 'Name must be 2–40 letters' and re-prompts without crashing.
FR-1	Name — valid	Given user enters 'Raj', when they submit, then system accepts and advances to phone input.
FR-2	Phone — starts with 1	Given user enters '1234567890', when they submit, then system shows 'Enter a 10-digit Indian mobile starting with 6, 7, 8, or 9' and re-prompts.
FR-2	Phone — valid	Given user enters '9876543210', when they submit, then system accepts and advances to quantity input.
FR-4	Quantity — word input	Given user enters 'three', when they submit, then system shows 'Quantity must be a whole number from 1 to 10' and re-prompts.
FR-4	Quantity = 11	Given user enters 11, when they submit, then system shows 'Maximum order is 10 pizzas per order' and re-prompts.

FR	Scenario	Given / When / Then
FR-5	File missing at startup	Given Types_of_Pizza.txt is missing when app starts, when app loads, then system shows 'Menu file Types_of_Pizza.txt not found. Please check and restart.' and exits before any user prompt.
FR-5	All files valid	Given all 3 files are present and valid, when app loads, then menu is in memory before the first user prompt appears.
FR-9	Discount — qty 5	Given customer selects qty 5, when bill is computed, then 'Discount (10%): -Rs.[amount]' appears between subtotal and GST.
FR-9	No discount — qty 4	Given customer selects qty 4, when bill is computed, then no discount line appears and GST is applied to full subtotal.
FR-17	Recommendation — returning	Given customer with 3 prior orders enters phone and submits, when system queries Supabase and gets results, then recommendation card shows pizza name, topping name, and 1-2 sentence reason before menu selection.
FR-18	Recommendation — new	Given customer with no prior orders in Supabase, when they submit phone, then recommendation step is skipped and menu loads directly with no error.
FR-19	Recommendation — fallback	Given OpenRouter returns an error, when recommendation step is triggered, then system shows 'We couldn't generate a suggestion — here's the full menu.' and proceeds within 3 seconds.
FR-20	Admin auth	Given unauthenticated user navigates to admin URL, when page loads, then they are redirected to Supabase Auth login and no order data is visible.
FR-23	CSV export	Given admin clicks 'Export CSV', when download completes, then a CSV is downloaded with all orders fields and a header row for the current filter.

A.8 User Flow Diagram

The flow below shows the complete ordering journey. **Key fix from v0.1:** menu files are loaded at app startup (before any user input), not mid-flow. The recommendation branch is Stage 3 only — in Stage 2 the flow goes directly from phone validation to menu selection.

```
APP STARTUP
    ■■■ Load all 3 menu files
          ■■■ Files missing/empty? → Show error + EXIT
          ■■■ Files OK → Log session timestamp
                ■■■ [Stage 3 only] Query Supabase for order history by phone
                      ■■■ History found? → Show AI recommendation → Accept / Skip
                      ■■■ No history? → Proceed directly to menu

CUSTOMER INTAKE
    Enter name (validate) → Enter phone (validate) → Enter quantity (validate)

MENU SELECTION
    Select base by number (validate)
    Select pizza by number (validate)
    Select topping by number (validate)

PRICING ENGINE
    Compute subtotal x qty
    Qty >= 5? → Apply 10% discount
    GST 18% on post-discount total
    Render itemised bill

PAYMENT
    Select Cash / Card / UPI (validate)
    Show mode-specific confirmation message
```

```
PERSISTENCE
    Stage 2: Append to orders_log.txt
    Stage 3: Write to Supabase orders + order_line_items

COMPLETE
    Order confirmed → New order? → YES: return to Customer Intake
                                 → NO:  Exit
```

A.9 Edge Cases — Five-Category Framework

Category	Scenario	Expected Behaviour
Empty State	New customer — no Supabase order history	Skip recommendation step silently; proceed to menu (FR-18).
Empty State	Admin dashboard — no orders in DB yet	Show 'No orders yet' placeholder; not a crash or blank table.
Empty State	Menu file has zero valid lines after parsing	Refuse to start; display which file is empty.
Empty State	orders_log.txt does not exist on first run	Create file; proceed normally.
Error State	OpenRouter returns null or HTTP error	Display fallback message per FR-19; proceed to menu; log failure.
Error State	OpenRouter times out (>5 seconds)	Same fallback; do not block the ordering flow.
Error State	Supabase write fails (Stage 3)	Display 'Order confirmed but could not be saved. Contact operator.' Do not crash.
Error State	Menu file missing at startup	Clear error naming the file; exit before any user prompt.
Error State	orders_log.txt write fails	Confirm order to customer; warn operator; do not crash.
Boundary Condition	Qty = 1 and Qty = 10	Both accepted. Qty 10 has no discount (threshold is >=5).
Boundary Condition	Qty = 5 exactly	Discount applies. Verify discount line appears on bill.
Boundary Condition	Item selection = 1 and = N (last)	Both accepted. 0 and N+1 rejected with range message.
Boundary Condition	Customer has exactly 1 prior order	Recommendation engine runs; 1 order is sufficient.
Concurrent Action	Two Gradio sessions write to log (Stage 2)	Known limitation. Documented in non-goals. Resolved in Stage 3 via Supabase.
Concurrent Action	Admin views orders while customer places one (Stage 3)	No conflict. Admin dashboard is read-only; Supabase handles concurrent writes.
Permission Violation	Non-admin accesses admin URL	Redirect to Supabase Auth login; no order data exposed.
Permission Violation	Admin attempts to edit/delete an order	No edit/delete controls on dashboard — read and export only.

A.10 Non-Goals / Out of Scope

Stage 2 deferrals that become Stage 3 goals:

- Owner query UI for orders → becomes admin dashboard in Stage 3.
- Concurrent session safety → resolved by Supabase in Stage 3.
- Menu in flat files → moves to DB tables in Stage 3.

Permanent non-goals through Stage 3:

- Real payment gateway — Cash/Card/UPI are confirmation modes only; no settlement, no refunds.

• Delivery address capture or zone validation.
• Inventory management — system does not check or decrement stock.
• Mixed-pizza orders — one base + one pizza + one topping x quantity N of the same combination per order.
• Customer-facing accounts or login — phone number is the only identifier.
• Multiple AI features — Option A only. Options B and C are out of scope.

A.11 Open Questions

#	Question	Why It Matters	Owner	Due
OQ-1	After order confirms in Gradio, reset for new customer or exit?	Determines whether session state needs a reset path.	Team	Before Stage 2
OQ-2	For new customers, show generic 'most popular' fallback or skip silently?	Affects FR-18 implementation and UX.	Team	Before Stage 3
OQ-3	Which OpenRouter model?	Cost, latency, quality tradeoff; must be in README for rubric.	Team	Before Stage 3
OQ-4	Can customer decline recommendation and browse full menu?	Determines whether recommendation pre-fills or replaces menu selection.	Team	Before Stage 3

A.12 Drawbacks Analysis

• **No concurrency / single-session state.** The Gradio MVP is not a multi-tenant queue. Two simultaneous customers can collide on shared state. Fine for a demo; not for live peak load.

• **orders_log.txt does not scale.** At 1,000+ orders: no indexing, slow analytics (full-file scans), no concurrent-write safety. This is why Stage 3 moves to Postgres/Supabase.

• **No real payment integration.** Payment is a confirmation mode only — no gateway, settlement, or refund path.

• **No address / delivery capture.** The flow takes name + phone, not delivery address or zone — yet delivery radius is the core operational constraint.

• **No inventory / availability.** Nothing prevents ordering a sold-out item.

• **No auth, no idempotency (Stage 2).** No order IDs, no protection against duplicate submission.

• **GST/discount rules are hard-coded.** A rate or threshold change requires a code edit, not a config change.

• **Single pizza configuration per order.** One-of-each model understates real basket complexity.

• **Recommendation cold start.** No data for new customers and no 'most popular' fallback unless OQ-2 is resolved before Stage 3 build.

A.13 Cost vs Value Analysis

Estimated build effort:

Stage	Scope	Estimated Effort
Stage 1	PRD + economics	8–12 hrs
Stage 2	Gradio MVP — validation, menu load, pricing, GST, persistence, 8 edge cases	20–28 hrs
Stage 3	Vercel + Supabase + admin dashboard + AI recommendation engine	45–60 hrs

Stage	Scope	Estimated Effort
Total	Full product	~75–100 hrs (team of 3–4, 3 weeks)

Measurable value to the outlet:

- **Throughput / lost-order recovery.** Recovering ~5 lost peak-hour orders/day at ~Rs.511 contribution margin protects ~Rs.76k/month in margin.
- **Labour leverage.** Self-serve intake frees the Rs.18k/month counter role for kitchen work.
- **Error elimination.** Deterministic pricing/GST removes remake-and-refund costs.
- **Data asset.** Structured order log unlocks BI levers (peak-hour staffing, AOV/upsell, retention).
- **AI upsell (Stage 3).** Recommendation engine targets topping attach rate — the highest-margin component (74–85% gross margin). Even 15% recommendation acceptance materially lifts AOV.

A.14 Funnel Metrics + Impact Metrics

Funnel Metrics (was the feature built correctly and can users get through it?)

1. % of sessions completing customer intake without abandonment.
2. % of sessions reaching payment selection after menu selection.
3. % of completed sessions successfully persisting to log / Supabase.
4. Error rate by input type — which prompt generates the most re-prompts.
5. % of returning customers shown a recommendation (Stage 3).
6. % of customers who received a recommendation and proceeded to order (engagement rate).

Impact Metrics (did the system achieve the business objective?)

1. Digital orders per day vs estimated phone baseline — are we replacing phone volume?
2. AOV in system vs Rs.847 reference baseline.
3. Recommendation acceptance rate and resulting AOV delta.
4. Pricing/GST correction calls post-launch — target: zero.

A.15 Version History

Version	Date	Change
v0.1	2026-06-23	Initial draft — Stage 2 scope only.
v0.2	2026-06-24	Added GWT acceptance criteria, 5-category edge case framework, non-goals, open questions.
v0.3	2026-06-24	Expanded to full Stage 3 scope. AI feature locked as Option A. Admin persona and dashboard FRs added. Supabase schema defined. Recommendation engine spec and system prompt written. User flow diagram fixed (files now load at startup). Session-notes frameworks applied throughout.

PART B — BUSINESS UNIT ECONOMICS

Baseline numbers are taken from *SliceMatic_Business_Economics.pdf* and **challenged where they don't hold up**. The rubric rewards internal consistency, not acceptance.

B.1 Monthly Fixed Costs

Cost Item	Monthly (Rs.)
Kitchen rent (1,200 sq ft)	55,000
Equipment EMI (Rs.4.5L, 36-mo)	14,500
Electricity	12,600
Gas / LPG	5,550
Head chef	28,000
Kitchen helper	16,500
Counter + billing	18,000
Delivery riders (2, fixed)	32,000
Internet + POS + SaaS	2,800
Marketing (fixed)	8,000
Packaging (fixed component)	4,200
Misc / contingency (3%)	5,760
TOTAL FIXED COSTS	2,02,910

B.2 COGS per Pizza + Category Margin Split

Component	Margherita	Farm House	Cheese Burst	Avg
Base ingredients	38	45	72	52
Sauce + seasoning	12	14	14	13
Cheese (mozzarella)	28	32	65	38
Pizza-specific toppings	8	22	18	17
Add-on topping (avg 1)	10	10	10	10
Packaging (box + bag)	18	18	18	18
Delivery variable cost	22	22	22	22
Total COGS / pizza	136	163	219	170

Gross margin % by menu category:

Category	Menu Price Band	Component Cost	Gross Margin
Base (crust)	Rs.149–229	Rs.38–72	~68–74%
Pizza (sauce + cheese + toppings)	Rs.299–379	~Rs.68	~77–82%
Add-on topping	Rs.39–69	~Rs.10	~74–85%

Toppings and the pizza layer are the highest-margin components — which is why the recommendation engine (FR-17) targets topping attach rate as its primary margin lever.

B.3 Revenue Model — 60% Capacity

Metric	Weekday	Weekend/Holiday	Monthly Total
Days in period	22	8	30
Orders per day (60% cap)	38	68	~47 avg
Average order value (AOV)	Rs.792	Rs.940	Rs.847
Daily revenue	Rs.30,096	Rs.63,920	~Rs.40,500 avg
Monthly gross revenue	Rs.6,62,112	Rs.5,11,360	Rs.11,73,472
GST collected (18%, passed to Govt.)	—	—	Rs.1,77,564
Net revenue (ex-GST)	—	—	Rs.9,95,908

B.4 Contribution Margin — Reference vs Corrected

The reference doc charges **one pizza's variable cost** against a **multi-pizza order's revenue** (AOV Rs.847 ≈ ~2 pizzas ex-GST, but only ~1 pizza's COGS is deducted). That overstates CM significantly.

P&L; Line Item	Reference Doc	Corrected (~2-pizza basket)
Revenue ex-GST	847	847
Less: Ingredient COGS	(148)	(~260)
Less: Packaging	(18)	(~36)
Less: Delivery variable (rider incentive)	(22)	(22)
Less: Payment gateway fee (1.8%)	(15)	(~18)
Contribution Margin	644 (76%)	~511 (~60%)

The doc's 76% CM is ~26% overstated. A realistic CM is **~60% (Rs.511/order)**. Everything downstream shifts with it.

B.5 Break-Even — Corrected

	Reference	Corrected
Fixed costs / month	2,02,910	2,02,910
CM / order	644	511
Break-even orders / month	315	~397
Break-even orders / day	11	~13

At 47 orders/day (60% capacity), SliceMatic runs at **~3.5x break-even** — healthy, but not the 4.3x the doc claims. The model is robust; the headline was optimistic.

B.6 Two Internal Inconsistencies in the Reference Document

1. Page-1 'Monthly Revenue Rs.2.1L (60% capacity)' is mislabelled. Rs.2.03L is total fixed cost, not revenue. Actual 60% revenue is Rs.11.73L gross / Rs.9.96L net.

2. Section 5's 58.9% operating margin contradicts the Section 8 ramp. Section 5 assumes instant 60% capacity from day one (EBITDA Rs.7.0L/month). The 18-month ramp shows Month 1 at 20% capacity / Rs.0.91L EBITDA (~23% margin). Section 5 is an idealised maturity snapshot; Section 8 is the realistic path. Quoting both as if they describe the same moment is misleading.

B.7 Delivery Radius Economics

Radius	Est. Households	Avg Delivery Time	Viable?
0–2 km	~18,000	8–12 min	Yes — premium SLA
2–4 km	~55,000	15–22 min	Yes — sweet spot
4–6 km	~1,10,000	25–40 min	Marginal — 3rd rider needed
> 6 km	Large	40+ min	Not viable at launch

Recommendation: launch at 4 km. Expand to 5 km after adding a 3rd rider (~+Rs.16k/month), justified at ~55 orders/day.

B.8 GST Treatment — Who Absorbs It?

The customer absorbs output GST; SliceMatic never books it as revenue. 18% GST on restaurant home-delivery is collected from the customer, held as a liability, and remitted monthly. The P&L; is always computed ex-GST. SliceMatic reclaims Input Tax Credit (ITC) (~Rs.18–22k/month) on GST paid for ingredients, packaging, and equipment. Net GST payable: ~Rs.1.78L – Rs.0.20L ITC ≈ **Rs.1.58L/month**.

Implementation rule: store menu prices exclusive of GST; add 18% on the post-discount subtotal at billing (matches FR-10).

Sample bill: 5 x (Cheese Burst Rs.229 + BBQ Chicken Rs.379 + Extra Cheese Rs.69) = Rs.3,385 → –10% discount Rs.338.50 → Rs.3,046.50 → +18% GST Rs.548.37 → **Total: Rs.3,594.87**

B.9 Stage-1 Challenge Questions — Analysis

Q1 — Rent rises to Rs.70,000. Fixed cost → Rs.2,17,910. Break-even at corrected CM Rs.511 → ~427 orders/month ≈ ~14/day (from ~13). Each Rs.10k of rent adds only ~0.6 orders/day. To push break-even past the 47/day target, fixed costs would have to exceed ~Rs.7.2L/month. The real risks are demand cold-start and the overstated CM, not rent.

Q2 — 40% of orders via aggregator at 25% commission. Aggregator CM ≈ 511 + 22 – 212 ≈ Rs.321. Blended CM = 0.6×511 + 0.4×321 ≈ Rs.435. New break-even ≈ ~466 orders/month ≈ 16/day (up from 13). Use aggregators for discovery; convert repeat customers to own channel — which is exactly what the Stage 3 recommendation engine is designed for.

Q3 — When is a 3rd rider justified? Financially trivial: Rs.16k / Rs.511 CM ≈ 32 extra orders/month — under 1/day pays for it. The binding trigger is throughput: two riders manage ~60–70 deliveries/day. Hire the 3rd rider when sustained daily volume approaches ~55 to protect delivery SLA.

Q4 — Is the qty≥5 / 10% discount a value-driver or a leak? On a 5-pizza order the 10% (Rs.338.50) comes straight out of contribution, cutting that order's CM by ~14%. **Verdict:** value-driver only if it changes behaviour (pulls 3–4 pizza baskets up to 5); a pure leak if customers who'd buy 5 anyway just pocket 10%. Fix: nudge at qty 4 — "add 1 more, save 10%" — so you discount incremental baskets, not inframarginal ones.

Q5 — COGS +12% inflation. Ingredient COGS +12% ≈ +Rs.36 on a 2-pizza order → corrected CM falls from ~Rs.511 to ~Rs.475. To hold the same monthly EBITDA: +2–3 orders/day, or a ~Rs.36/order price rise. Manageable — but pair it with menu re-engineering toward the high-margin pizza/topping layer.

Q6 — Three BI metrics from the order DB that improve profitability:

1. **Peak-hour heatmap (orders x hour x day-of-week)** → staffing, prep batching, 3rd-rider timing. Maps to Stage 3 admin dashboard (FR-22).
2. **Basket / attach analysis (AOV, topping attach rate, discount uptake)** → menu engineering and targeted upsell. Maps to Stage 3 recommendation engine (FR-17).
3. **Repeat-rate / RFM by phone number** → retention and own-channel conversion. Protects CM from aggregator commission and lowers acquisition cost.